# Geometry-Aware Deep Recurrent Neural Networks for Hyperspectral Image Classification

Siyuan Hao, *Member, IEEE*, Wei Wang, and Mathieu Salzmann, *Member, IEEE*

***Abstract*— Variants of deep networks have been widely used for hyperspectral image (HSI)-classification tasks. Among them, in recent years, recurrent neural networks (RNNs) have attracted considerable attention in the remote sensing community. However, complex geometries cannot be learned easily by the traditional recurrent units [e.g., long short-term memory (LSTM) and gated recurrent unit (GRU)]. In this article, we propose a geometry-aware deep recurrent neural network (Geo-DRNN) for HSI classification. We build this network upon two modules: a U-shaped network (U-Net) and RNNs. We first input the original HSI patches to the U-Net, which can be trained with very few images and obtain a preliminary classification result. We then add RNNs on the top of the U-Net so as to mimic the human brain to refine continuously the output-classification map. However, instead of using the traditional dot product in each gate of the RNNs, we introduce a Net-Gated GRU that increases the nonlinear representation power. Finally, we use a pretrained ResNet as a regularizer to improve further the ability of the proposed network to describe complex geometries. To this end, we construct a geometry-aware ResNet loss, which leverages the pretrained ResNet's knowledge about the different structures in the real world. Our experimental results on real HSIs and road topology images demonstrate that our approach outperforms the state-of-the-art classification methods and can learn complex geometries.**

***Index Terms*— Deep learning, geometry-aware loss, gated recurrent unit (GRU), hyperspectral image (HSI) classification, Net-Gated recurrent neural networks (RNNs), remote sensing, U-shaped network (U-Net).**

## I. Introduction

With the rapid development of the sensor technology, intelligent analysis of a hyperspectral image (HSI) has been widely used in agriculture, forestry, geology, star exploration, military, and other fields. Image classification is an important branch of intelligent HSI analysis, which has attracted considerable attention in the remote sensing community. Many effective classification methods have been investigated, such as random forests [1] and support vector machine (SVM) [2], [3]. However, these methods cannot be trained in an end-to-end manner, and thus rely on the handcrafted features instead of unifying the feature-extraction and classification stages.

By contrast, the features extracted by the deep networks—deep features—can be more discriminative than the handcrafted ones for HSI classification [4]. The first attempt can be found in [5], where stacked autoencoders (SAEs) were introduced to extract the deep and hierarchical features. These features, however, were limited to the spectral domain. Considering the great contribution of spatial information to HSI classification, Tao *et al.* [6] adaptively learned the spectral–spatial features by a stacked sparse autoencoder, and Ma *et al.* [7] proposed to add the spectral and spatial constraints in the loss function of a traditional SAE. Nevertheless, all the networks mentioned above only consist of fully connected layers, which are more expensive to train than the convolutional ones. Convolutional neural networks (CNNs) are locally connected neural networks with shared weights, which greatly reduce the number of the network parameters and the complexity of the networks. Therefore, many variants of the CNN models were introduced for image classification [8]. For instance, Hu *et al.* [9] treated each HSI pixel as a 2-D image (whose height is equal to 1) and simply employed a 1-D-CNN to classify these HSI pixels. This, however, again discards the spatial information contained in the original HSI. To address this, the 2-D-CNNs [10], [11], 3-D-CNNs [12]–[14], and pyramidal residual networks (p-ResNets) [15] were employed. They consider the spatial information and can generally exploit the mainstream networks. Nonetheless, while many deeper CNNs have been constructed to work with the RGB images, such as AlexNet [16], VGG-Net [17], GoogLeNet [18], and ResNet [19], their application to HSIs remains limited, because these networks usually require thousands of annotated training samples, which goes beyond the typical data available for the HSI tasks.

Many strategies have been presented to learn the network parameters without a large number of annotated training samples, such as unsupervised learning and transfer learning. In the former case, Hinton *et al.* introduced deep belief networks (DBNs), which can be trained in a purely unsupervised manner and Romero *et al.* [21] used an unsupervised deep CNN to search for the sparse features for HSI classification. In the latter, the transfer learning theory has shown that the network parameters learned from a source domain can be transferred to a target one by fine-tuning. To this end, a two-channel deep CNN was built in [22]; Hao *et al.* [23] pretrained a

Manuscript received October 4, 2019; revised February 13, 2020 and May 18, 2020; accepted June 13, 2020. Date of publication July 10, 2020; date of current version February 25, 2021. This work was supported in part by the National Natural Science Foundation of China under Grant 61701272. *(Corresponding author: Siyuan Hao.)*

Siyuan Hao is with the College of Information and Control Engineering, Qingdao University of Technology, Qingdao 266520, China (e-mail: lemonbananan@163.com).

Wei Wang is with Computer Vision Laboratory, École polytechnique fédérale de Lausanne (EPFL), 1015 Lausanne, Switzerland (e-mail: wei.wang@epfl.ch).

Mathieu Salzmann is with Computer Vision Laboratory, École polytechnique fédérale de Lausanne (EPFL), 1015 Lausanne, Switzerland, and also with ClearSpace, 1024 Ecublens, Switzerland (e-mail: mathieu.salzmann@epfl.ch).

Color versions of one or more of the figures in this article are available online at https://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TGRS.2020.3005623

0196-2892 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

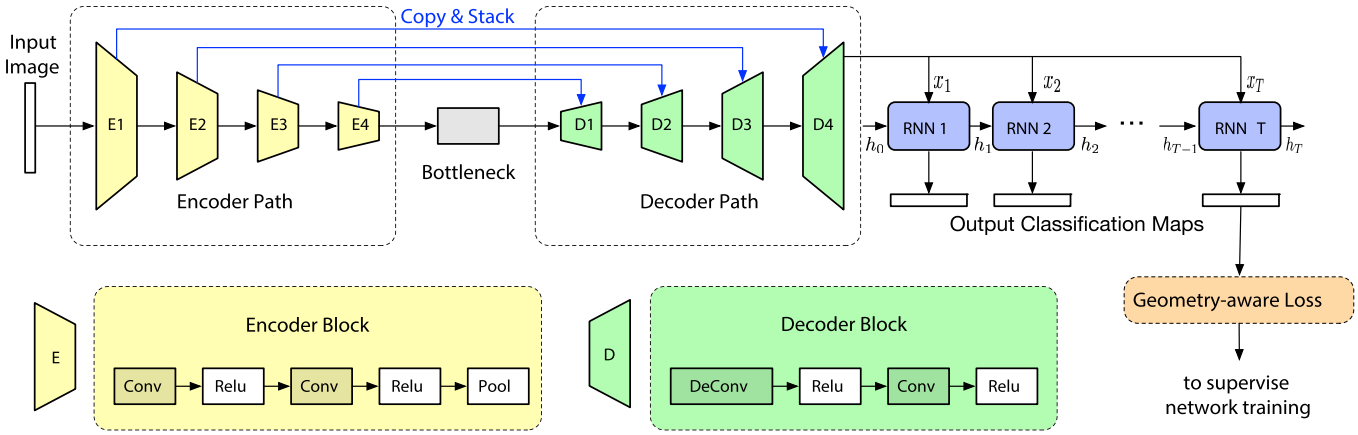



Fig. 1. Proposed Geo-DRNN framework. U-Net provides a preliminary classification result, which is continuously refined by the subsequent RNNs. U-Net is composed of an encoder, a bottleneck, and a decoder, and the internal compositions of the encoder and decoder blocks are shown at the bottom of the figure.

super-resolution CNN on an auxiliary domain to solve the problem of data limitation. Moreover, Mukhometzianov and Carrillo [24] investigated the use of capsule networks (CapsNets), which also do not require a large number of training samples to generalize well. Apart from the above works, the U-shaped net (U-Net) also benefits from not requiring thousands of samples for training [25], because its skip connections enable it to retain the details in the original image by passing the features from the encoder to the decoder. Such encoder information has been shown to be helpful for the final prediction. U-Net is very shallow and compact, with a limited amount of parameters. Thus, it has a low risk of overfitting to small data sets. As observed empirically in [26] and [27], U-Net yields excellent performance in biomedical image segmentation, for which large amounts of annotated data are typically hard to come by. Since the HSI shares this property of limited access to the annotated training data, we choose U-Net as the backbone of our approach. In practice, the input images of U-Net are usually of relatively large sizes, that is, the input images include rich topological information and complex geometries, which can help to boost the classification performance. Effectively leveraging this information, however, remains a challenge.

In the field of remote sensing, each hyperspectral pixel can be viewed as a set of ordered channels and a continuing sequence in the spectral space [28], [29]. The CNNs process all channels in the spectra sequences at once, that is, they take as input the entire patches with all the channels, process it, and return an output. The dependences between the channels are implicitly modeled. In contrast to the CNNs, RNNs are employed to learn the dependences between the channels in the spectra sequences explicitly. For the input image, the channels are partitioned into a sequence of smaller groups, and at each recurrence, one group of channels is fed to the RNNs. This allows the network to model explicitly the dependences between the channel groups. Inspired by this, Mou *et al.* [30] first introduced the use of traditional RNNs for HSI classification, and Wu and Prasad [31] improved these traditional RNNs to convolutional ones to learn more discriminative spectral features. The RNNs can also be used to extract features along different dimensions. For example, Liu *et al.* [32] designed bidirectional-convolutional long short-term memory (LSTM) networks to extract the spectral–spatial features, and Rußwurm and Körner [33], Benedetti *et al.* [34], and Mou *et al.* [35] successively proposed to use convolutional RNNs to extract the joint spectral–spatial–temporal features. Furthermore, Yang *et al.* [36] found that multiscale recurrent models always outperform the traditional deep 2-D or 3-D models. In fact, the model in [36] shares the same motivation of recursively refining the prediction as that in this article. However, similar to [37], the recurrent model in [36] is a naive recurrent network without any gate or memory.

Nevertheless, from a biological perspective, we human neither understand images at one glance (like CNNs) nor process different spectral channels individually to learn the dependences afterward (like RNNs). For example, when we read, we understand each word based on our memory of its previous context. In other words, humans do not need to start their thinking from scratch. We have a memory of previous understandings, and we have the ability to refine it.

To mimic the human vision system, in this article, we use both CNNs and RNNs in a different way, as shown in Fig. 1. First, we use a CNN to preprocess the images, so as to play the role of the human eyes. We then obtain predictions based on the signals passed from the CNN and gradually refined these predictions using the RNNs supervised by the ground-truth labels at each recurrence. Note that, here, the RNNs are used to refine the predictions, not to learn the dependences between the channels. With the help of the RNN memory cells, the knowledge learned at different stages can be accumulated and used to refine the predictions continually. In other words, here, the RNNs play the role of human brain. However, the output of the above network does not enforce constraints on the geometric information. Ledig *et al.* [38] pointed out that a loss function based on the high-frequency details (such as geometric information) is closer to perceptual similarity.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

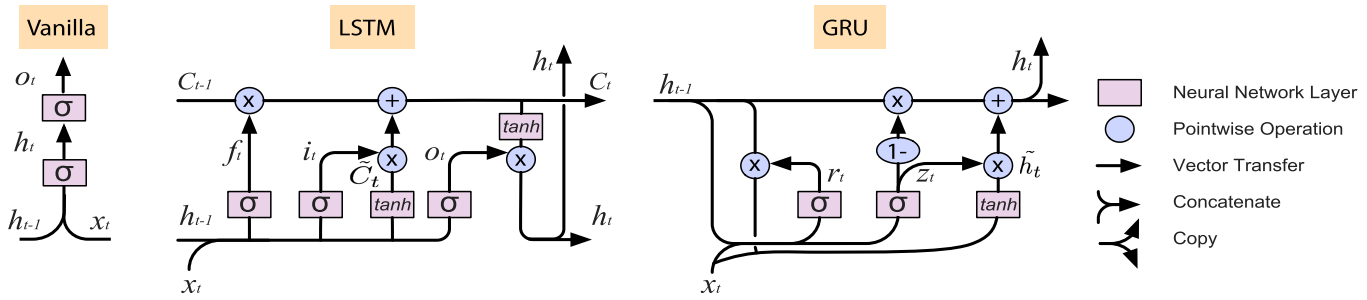



Fig. 2. Representative recurrent units. (a) Vanilla unit. (b) LSTM unit. (c) GRU unit.

Our brain has prior knowledge on object shapes. When we observe things in the real world, we have the ability to make better predictions based on our geometric priors. To mimic this process, we exploit the observation that, after passing an image through several convolutional layers, the receptive field of each neuron becomes larger, therefore providing feature maps in the middle of a deep network (e.g., ResNet and VGG19) with a global view of the image, encoding geometric information. To this end, we further append a pretrained ResNet to the RNNs, from which we design a loss function that exploits the geometric priors of the pretrainednetwork.

Furthermore, Rustowicz *et al.* [39] have verified that the combination of a convolutional LSTM (CLSTM) and a U-Net achieves good performance for satellite image classification, which validates our design. We, however, add the RNNs on the top of the U-Net instead of embedding a CLSTM within the U-Net. Specifically, we rely on the standard U-Net as the backbone to encode the original HSI. We then add several gated recurrent units (GRUs) to mimic the human brain and refine continuously the classification map. Finally, we input the resulting classification map and the corresponding ground truth into a pretrained ResNet to calculate a Geometry-aware ResNet loss, which helps to train the whole network. Our main contributions are as follows.

1) While the existing literature exploits RNNs to model the HSI channel dependences from the perspective of a continuing spectra sequence, we use RNNs from the perspective of biological perception to refine the predictions recursively. The RNNs are located on the top of a U-Net whose purpose is to extract spatial information to facilitate prediction.
2) To make full use of rich topological information and complex geometries, we introduce a Net-Gated GRU, whose gates are implemented by a set of small networks that increase the nonlinear representation power to extract the high-level representation of complex geometries.
3) We propose a geometry-aware ResNet loss to compensate for the limitations of the traditional cross-entropy loss. Our loss exploits the knowledge contained in a pretrained ResNet to better encode the complex geometries.

The rest of this article is organized as follows. Section II briefly reviews the background of the RNNs. Section III introduces our method. Section IV evaluates its effectiveness using hyperspectral data sets and road images. Section V analyzes the experimental results and draws the conclusions.

## II. Background of Recurrent Neural Networks (RNNs)

RNNs allow us to operate on a sequence of vectors, in which the recurrent architecture can propagate previous information to the present unit (i.e., implemented by the hidden state vectors). Three representative recurrent units are visualized in Fig. 2, which are, Vanilla unit (**Vanilla**) on the left, LSTM networks in the middle, and GRU on the right.

Let $\mathbf{x} = (x_1, x_2, \ldots, x_t, \ldots, x_T)$ be an input sequence, where $x_t$ is the input at the $t$th recurrence. $\mathbf{h} = (h_0, h_1, \ldots, h_t, \ldots, h_T)$ and $\mathbf{o} = (o_1, o_2, \ldots, o_t, \ldots, o_T)$ are the hidden state sequence and the output sequence, respectively. Vanilla is the simplest recurrent unit, capable of processing the data with unlimited sequence lengths using the following nonlinear mapping:

$$h_t = \sigma\left(\mathbf{W} \cdot \left[h_{t-1}, x_t\right] + b\right) \tag{1}$$

where $\mathbf{W}$ is the affine transformation matrix and $b$ denotes the bias. $\sigma$ is the nonlinear activation function (e.g., sigmoid or hyperbolic function). We initialize the network at the first recurrence as $h_0 = 1$.

Vanilla RNNs, however, are difficult to train due to gradient vanishing and exploding problems, especially for long input vectors [40], [41]. Therefore, LSTM networks that are explicitly designed to tackle the long-term dependence problem [42] were proposed. The key idea behind them is their memory state and four gates capable of regulating and controlling the information flow inside the unit. The four control gates are defined as follows:

$$\begin{aligned} f_t &= \sigma\left(\mathbf{W}_f \cdot \left[h_{t-1}, x_t\right] + b_f\right) \quad \text{(forget gate)} \\ i_t &= \sigma\left(\mathbf{W}_i \cdot \left[h_{t-1}, x_t\right] + b_i\right) \quad \text{(input gate)} \\ \tilde{C}_t &= \tanh\left(\mathbf{W}_C \cdot \left[h_{t-1}, x_t\right] + b_C\right) \quad \text{(modulation gate)} \\ o_t &= \sigma\left(\mathbf{W}_o \cdot \left[h_{t-1}, x_t\right] + b_o\right) \quad \text{(output gate).} \end{aligned} \tag{2}$$

The memory-state vector is defined as

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{3}$$

determining the amount of old memory that should be discarded and the amount of new memory that should be added. $\odot$ is the elementwise product. One can obtain the hidden state vector at the present recurrence as

$$h_t = o_t \odot \tanh(C_t)\ . \tag{4}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

The GRU is a simplified version of the LSTM [43]. Its structure is expressed as follows:

$$\begin{aligned} z_t &= \sigma\left(\mathbf{W}_z \cdot \left[h_{t-1}, x_t\right]\right) \quad \text{(update gate)} \\ r_t &= \sigma\left(\mathbf{W}_r \cdot \left[h_{t-1}, x_t\right]\right) \quad \text{(reset gate)} \\ \tilde{h_t} &= \tanh\left(\mathbf{W}_h \cdot \left[r_t \odot h_{t-1}, x_t\right]\right) \quad \text{(candidate)} \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h_t} \quad \text{(hidden state)} \end{aligned} \tag{5}$$

where $z_t$ decides the amount of memory from the hidden state to be updated by the candidate hidden state. $r_t$ decides the amount of values from the previous hidden state that should be ignored to compute the candidate. $h_t$ is the output hidden state of the GRU and $\tilde{h_t}$ is the candidate.

A GRU is computationally less expensive than an LSTM as it has fewer gates. However, the spatial connectivity between the pixels cannot be preserved through the dot product in (5), because the feature tensor needs to be flattened into a vector. As a consequence, the spatial information is lost. Therefore, to convert a gated architecture into a convolutional one, the dot products should be replaced with convolution operations [44]. The resulting convolutional GRU is defined as

$$\begin{aligned} z_t &= \sigma\left(\mathbf{W}_z * \left[h_{t-1}, x_t\right]\right) \quad \text{(conv. update gate)} \\ r_t &= \sigma\left(\mathbf{W}_r * \left[h_{t-1}, x_t\right]\right) \quad \text{(conv. reset gate)} \\ \tilde{h_t} &= \tanh\left(\mathbf{W}_h * \left[r_t \odot h_{t-1}, x_t\right]\right) \quad \text{(conv. candidate)} \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h_t}\ . \end{aligned} \tag{6}$$

## III. Proposed Method

The proposed classification framework is depicted in Fig. 1. It builds upon U-Net and RNNs. The U-Net undertakes to encode the original HSI to obtain a feature map. The RNNs mimic the human brain to refine the classification result continuously. Each GRU of the RNNs takes as input the feature map of the U-Net and the memory from the previous recurrence. The pretrained ResNet is then employed to enable the proposed architecture to describe the complex geometries. In the following sections, we introduce and analyze the details of our approach. Our core implementation is publicly available at https://drive.google.com/drive/folders/1AeF9q75H-N4RZr-19zO855hZ8lcNkmmys?usp=sharing.

### A. U-Net

The U-Net consists of a fully convolutional network comprising an encoder path, a bottleneck, and a symmetric decoder path, as shown in Fig. 1.

The encoder path contains four $\mathbf{E}_l$ ($l \in \{1, 2, 3, 4\}$) blocks, each of which is composed of a $2\times2$ max pooling and two $3\times3$ convolutions, each followed by a Relu. Let $\mathbf{s} \in \mathbb{R}^d$ represent a $d$-channel pixel in the HSI and $\mathbf{S} \in \mathbb{R}^{d\times m\times m}$ denote the image patch centered at $\mathbf{s}$. The feature map $\mathbf{E}_l(\mathbf{S})$ of the encoder path can be written as

$$\mathbf{E}_l(\mathbf{S}) = \text{Pool}\left(\left\{\text{Relu}\left(\mathbf{E}_{l-1}(\mathbf{S}) * \mathbf{W}_l^{\text{Conv}} + \mathbf{B}_l^{\text{Conv}}\right)\right\}_2\right) \tag{7}$$

where "$*$" denotes the convolution, Pool$(\cdot)$ is the max pooling, and $\{\cdot\}_2$ indicates that the procedure in parentheses is executed twice. $\mathbf{W}_l^{\text{Conv}}$ and $\mathbf{B}_l^{\text{Conv}}$ represent the kernels and biases of the $l$th encoder block, respectively.

The decoder path contains four $\mathbf{D}_l$ ($l \in \{1, 2, 3, 4\}$) blocks, each of which includes a $2 \times 2$ deconvolution, a skip connection with the corresponding cropped feature map from the encoder path, and two $3 \times 3$ convolutions, each followed by a Relu. The feature map $\mathbf{D}_l(\mathbf{S})$ for the $l$th block is

$$\mathbf{D}_l(\mathbf{S}) = \text{DeConv}\left(\left\{\text{Relu}\left(\left(\mathbf{E}_{l-1}^{\text{crop}}(\mathbf{S}) \oplus \mathbf{D}_{l-1}(\mathbf{S})\right) * \mathbf{W}_l^{\text{DeC}} + \mathbf{B}_l^{\text{DeC}}\right)\right\}_2\right) \tag{8}$$

where $\mathbf{E}_{l-1}^{\text{crop}}(\mathbf{S})$ is the cropped feature map transferred from the encoder path, whose size is the same as $\mathbf{D}_{l-1}(\mathbf{S})$. $\oplus$ is defined as the concatenation operation. DeConv$(\cdot)$ denotes the deconvolution. $\mathbf{W}_l^{\text{DeC}}$ and $\mathbf{B}_l^{\text{DeC}}$ represent the kernels and biases of the $l$th decoder block, respectively. Note that $\mathbf{E}_l(\mathbf{S})$ is doubled after each max pooling, but $\mathbf{D}_l(\mathbf{S})$ should be halved after each deconvolution. The center bottleneck simply consists of two $3 \times 3$ convolutions, each followed by a Relu.

In this architecture, the encoder path is in charge of the capturing context, while the decoder path enables precise localization. The context information is transferred to the decoder path through the skip connections, which improve the final performance. Furthermore, the experiments in [26] have demonstrated that the U-Net can be trained end to end with very few training images, which motivated our use of this architecture as the backbone for our method.

### B. Net-Gated GRU

The convolutional GRU can be implemented by feeding the input into a convolutional layer. Taking the convolutional update gate [in (6)] as an example, the concatenation of $h_{t-1}$ and $x_t$ is taken as the input to a convolutional layer with parameters $\mathbf{W}_z$, followed by a sigmoid function $\sigma(\cdot)$. However, passing the input through a single convolutional layer cannot approximate complex functions. Therefore, we extend this gate structure to **a small network** composed of a convolutional layer, a Relu layer, and another convolutional layer. The Relu layer increases the nonlinear representation power to approximate the complex functions, thus allowing us to extract more abstract and informative features. These features are high level and more suitable for representing complex geometries. We name this gate structure Net-Gated RNNs.

Note that, while deeper gated network could be used to extract even more abstract and informative features, they lead to a sharp increase in computational cost. In our experiments, we will explore the exact number of layers to include in the gate network.

### C. Geometry-Aware ResNet Loss

Let $\mathbf{S} \in \mathbb{R}^{d\times m\times m}$ be an HSI patch input to our geometry-aware deep recurrent neural networks (Geo-DRNNs) method and $f$ be the U-Net parameterized by weights $\mathbf{w}$, whose output can be expressed as $f(\mathbf{S}, \mathbf{w})$. We assume that the RNNs consist of $T$ GRUs, and the hidden state of the first GRU is updated as

$$h_t^1 = \begin{cases} 1, & \text{if } t = 0 \\ \sigma\left(h_{t-1}^1, f(\mathbf{S}, \mathbf{w})\right), & \text{otherwise} \end{cases} \tag{9}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

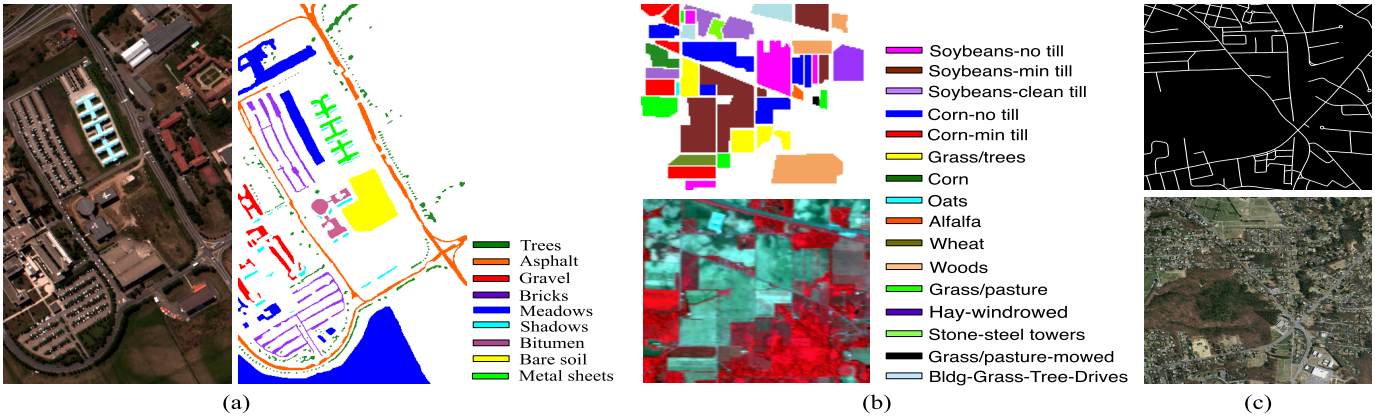



Fig. 3. False-color composition and reference land cover for small data sets. (a) Pavia University data set. (b) Indian Pines data set. (c) Massachusetts Roads data set (in which white represents the road and black the background).

where $h_{t-1}^1$ is the hidden state of the first GRU at the previous recurrence. Then, we obtain the classification map of this unit by inputting the above hidden state into a softmax layer, defined as

$$P^1\left(\hat{y} = j \mid h_t^1\right) = \frac{e^{h_t^1 w_j}}{\sum_{k=1}^{K} e^{h_t^1 w_k}} \tag{10}$$

where $w$ are the weights of the softmax layer and $K$ is the number of classes for the task of interest. The cross-entropy loss of this unit is denoted as $\ell_1$. By analogy, we can get the cross-entropy loss for other GRUs, i.e., $\ell_2, \ell_3, \ldots, \ell_T$. We train the whole network using the sum of the cross-entropy loss of each unit ($\ell_{\text{ce}}$) instead of employing the loss of the last one

$$\ell_1 = -\sum_{i=1}^{N} y_i \log(\hat{y}_i), \quad \ell_{\text{ce}} = \sum_{i=1}^{T} \ell_i \tag{11}$$

where $N$ is the number of training samples. $\hat{y}_i$ and $y_i$ are the predicted result and the ground truth, respectively.

Unfortunately, the widely used cross-entropy loss is local. It penalizes all the mistakes equally without considering any geometric information. To represent complex geometries, we employ the pretrained blocks of a residual neural network (ResNet). The blocks that come earlier in the ResNet extract local and highly generic feature maps (such as visual edges and colors), while the blocks higher up extract more abstract or semantic concepts (such as "cat ear" or "dog eye"). Therefore, we only use the first few blocks of the ResNet to do feature extraction, rather than using the entire convolutional layers. Here, the ResNet is trained with the ImageNet data set [45], which encompasses many different real-world geometries, that is, the pretrained ResNet can be used to represent the complex geometries.

Let us now introduce our geometry-aware ResNet loss. Specifically, we input the ground truth and the corresponding predicted classification map into the first few pretrained ResNet blocks, separately. We then add the resulting feature maps and define our geometry-aware ResNet loss as

$$\ell_{\text{geo}}(y, \hat{y}) = \sum_{n=1}^{B} \frac{1}{M_n W_n H_n} \sum_{m=1}^{M_n} |F_n^m(y) - F_n^m(\hat{y})| \tag{12}$$

where $F_n^m(\cdot)$ represents the $m$th feature map after passing the input image into the $n$th blocks of the pretrained ResNet and $B$ is the number of blocks considered in the ResNet. $M_n$ is the number of channels in the feature map, whose width and height are $W_n$ and $H_n$. The Geometry-aware ResNet loss is based on the abstract high-level feature maps, which better encode the complex geometries. To consider both simple linear structures and complex geometries, we combine the cross-entropy loss with the geometry-aware one. This yields the joint loss

$$\ell(y, \hat{y}) = \ell_{\text{ce}}(y, \hat{y}) + \lambda \ell_{\text{geo}}(y, \hat{y}) \tag{13}$$

where $\lambda$ is a parameter to balance the two terms.

### D. Training

The whole data set is divided into three parts: the training set, the validation set, and the test set. We randomly initialize the parameters of the U-Net and the RNNs while keeping the parameters of the ResNet frozen. This is because we do not seek to train further the pretrained ResNet, but rather to use it to extract the geometric information and compute the loss. Then, we input the training images and use the joint loss of (13) to train the U-Net and the RNNs. The validation set is used to select the best model. Finally, we report the final performance on the test set.

## IV. Experimental Results

### A. Data Set Description and Performance Evaluation

To demonstrate the effectiveness of our method, we make use of small data sets, including Pavia University, Indian Pine and Massachusetts Roads, and the large Houston University data set. Their related false-color compositions of the images and the reference land-cover maps are shown in Figs. 3 and 4. These data sets are described in more detail in the following.

*1) Small Data Sets:* Pavia University was obtained using the Reflective Optics System Imaging Spectrometer (ROSIS) sensor during a flight campaign over the Engineering School at University of Pavia, Italy. The spatial size of the images is $610 \times 340$ pixels, and 103 spectral bands are retained after

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

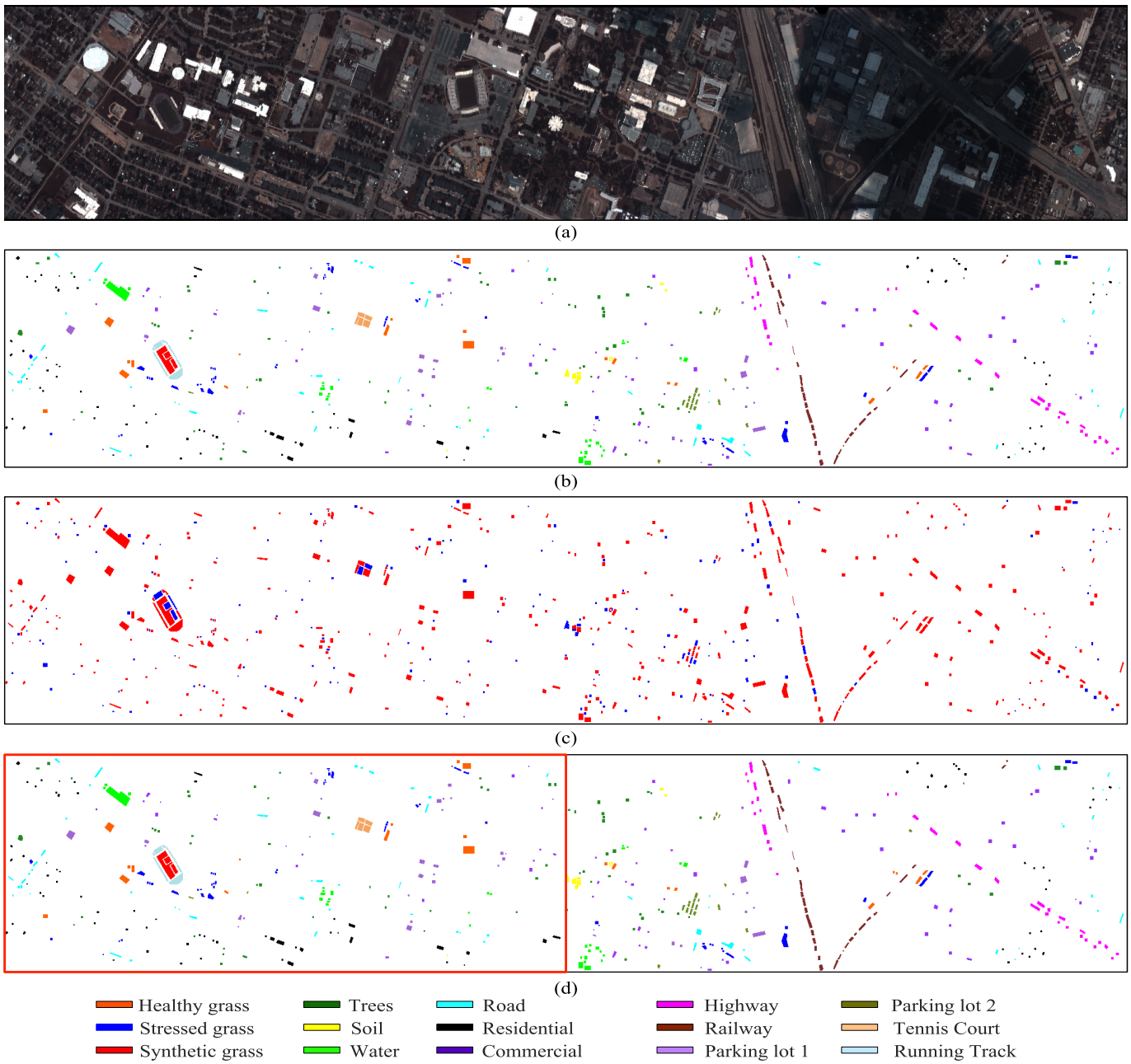



Fig. 4. Large Houston University data set. (a) False-color composition. (b) Reference land cover. (c) Training/test sample selection provided by the DASE Website (in which the training and test samples are shown in red and blue, respectively). (d) Disjoint training/test samples (in which the training samples are selected from the left region, while the test samples are from the right region).

discarding the effect of noise and water absorption. It includes nine classes, i.e., asphalt, meadows, trees, metal sheets, bare soil, bitumen, bricks, shadows, and gravel.

Indian Pines was collected by the AVIRIS sensor over Northwestern Indiana in June 1992. This image contains 220 spectral bands and has a spatial size of $145 \times 145$ pixels, with each pixel measuring approximately 20 m $\times$ 20 m on the ground. Twenty spectral bands were removed due to the noise and water-absorption phenomena. Sixteen mutually exclusive classes are included in this image.

Massachusetts Roads [46] is the largest publicly available aerial road data set, containing 1171 aerial images of the state of Massachusetts. It covers a wide range of urban, suburb, and rural regions. The corresponding road topology structure is complex. Each image contains $1500 \times 1500$ pixels, covering an area of 2.25 km$^2$. In this article, we used this data set to visualize the advantages of the proposed geometry-aware loss when classifying complex topologies.

*2) Large Data Set:* Houston University was acquired by the Compact Airborne Spectrographic Imager (CASI) sensor over an urban area nearby the campus of the University of Houston. This data set was proposed as a challenge for the IEEE IADF Data Fusion Contest 2013 [47]–[49]. It contains 144 spectral bands at 2.5-m resolution and has a spatial size

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

of 349 × 1905 pixels. Fifteen urban land-use class labels are available in this image. We split the training and testing samples following the Data and Algorithm Standard Evaluation (DASE) benchmarking Website [4], [50], as shown in Fig. 4(c). Note that this strategy results in some overlap between the training and testing samples. Therefore, in addition to this standard strategy, we also report the results on another disjoint split of the data, as illustrated in Fig. 4(d), where the training and testing splits have no overlap. Note that the training and testing samples are not selected randomly.

The overall accuracy (OA), the average accuracy (AA), the mean recall, and the mean F1 score are used to evaluate the performance. To avoid the biased estimates, ten independent tests were carried out using Pytorch on a computer equipped with an Intel Core i5 Processor at 2.70 GHz.

### B. Investigation of the Architecture of Geo-DRNNs

The proposed Geo-DRNNs is a combination of U-Net and RNNs, and then uses a pretrained ResNet as an extractor to calculate the perceptual loss to supervise the entire training process. As such, several factors affect the final performance. In this section, we explore how the performance is affected by these factors.

We start from the U-Net architecture in our Geo-DRNNs. As described in Section III-A, the ratio of the training images in the total images (denoted by $r$) and the ratio of the spatial size of the input patches (denoted by $s$) are two important factors to affect the final performance. Note that after each pooling operation with a stride of 2, the width and height of the tensor will be decreased by a factor 2. Therefore, the parameter $s$ must be large enough due to the max pooling and deconvolution in the U-Net, and the minimum log value of $s$ should be in direct proportion with the number of pooling layers. First, we cropped the input patches from the original HSI and kept the number of spectral bands constant. Then, we made $r$ and $s$ vary in the sets {1, 3, 5, 10, 15, 20, 30, 40, 50, 60} and {8 × 8, 16 × 16, 32 × 32}, respectively. The parameter setting for U-Net is as follows: batch size = 16, learning rate = 1$e$-5, and the numbers of the filters for the encoder and decoder blocks are set to [32, 64, 128, 256, 512]. Furthermore, to avoid a sharp decline in the spatial size of the input image, we set the kernel size to 3, the stride to 1, and the padding to 1 for each convolutional layer in each encoder/decoder block. If $s = 8 \times 8$, we set the parameter $l$ to 3 [see (7) and (8)]; otherwise, we use $l$= 4. The experimental results are summarized in Fig. 5, from which we can see that the U-Net achieves competitive accuracy even when using very few training images (OA ≥ 90%). Moreover, on average, the OA of $s = 16 \times 16$ is higher than those obtained using $s = 8 \times 8$ and $s = 32 \times 32$. Therefore, we fix $s$ to 16 × 16 in the following experiments. We also conducted this test on the Houston University data set, which has a larger coverage than Pavia University. We split the training and test samples, as shown in Fig. 4(c), and studied the impact of the spatial size of the input patches $s$, varying in {8 × 8, 16 × 16, 32 × 32}. We first assume that the number of GRUs is three. The experimental results are summarized in Table I. As for the Pavia University data set, the best accuracies are achieved for $s = 16 \times 16$.

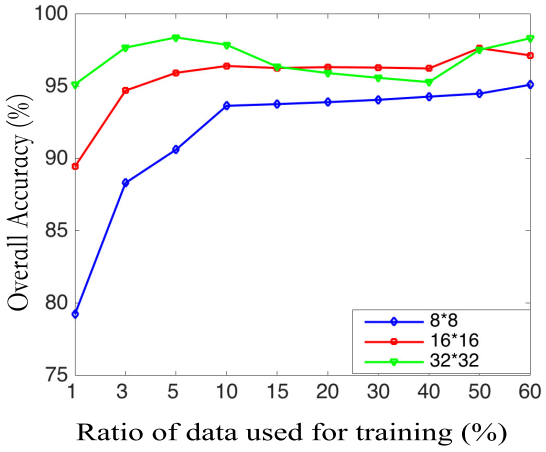



Fig. 5. Overall classification accuracies influenced by different training ratios and input spatial sizes on Pavia University.

TABLE I
IMPACT BY INPUT SPATIAL SIZES ON HOUSTON UNIVERSITY

| Performance (%) | $s = 8 \times 8$ | $s = 16 \times 16$ | $s = 32 \times 32$ |
|---|---|---|---|
| Mean Recall | 92.01 | **92.29** | 83.94 |
| Mean f1score | 92.09 | **92.84** | 86.62 |
| OA | 94.56 | **95.76** | 92.53 |
| AA | 86.54 | **86.90** | 78.15 |

In addition to validating the two parameters discussed above, we also study the gradients backpropagated through the various U-Net layers. Fig. 6 shows the histogram of the gradients at each layer in U-Net, where each slice represents the exponential distribution of the gradients. The slices in each subfigure are organized by the epochs along the vertical axis; the initial slice (i.e., epoch 0) is located in the background, while the last slice (i.e., epoch 5938) is in the foreground. Noting that darker shades represent larger gradients, these results indicate that the gradients become increasingly smaller as training progresses.

Regarding the RNNs, we explore the impact on the performance of the number of GRUs and the depth of the proposed small network embedded in each GRU. Fig. 7 and Table II show the corresponding experimental results on the Pavia University and Houston University data sets. To analyze better the impact of these parameters, we perform tests under relatively stable conditions. For the Pavia University data set, if the amount of training images is too small, there will be a high probability of overfitting. Therefore, we selected 60% cropped images as the training data for Pavia University. For the Houston University data set, we still selected the training and test data according to Fig. 4(c). We initialize the hidden state vector $h_0 = 1$ and set its size to 32. When we train the network, we set the batch size to 16, the learning rate to 1$e$-2, and made the number of GRUs vary from 0 to 5. From Fig. 7, we can observe that the performance is improved by adding the GRU architecture on the top of the U-Net. For the Pavia University data set, the performance increases with the number

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

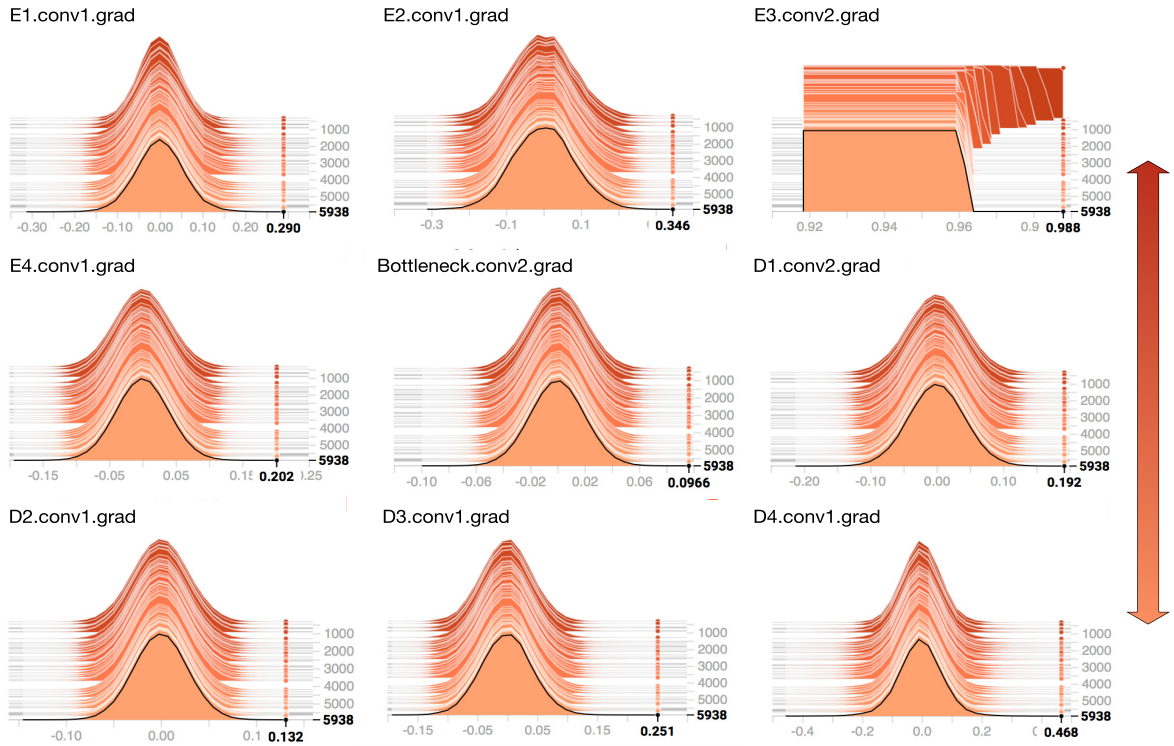



Fig. 6. Histogram of the gradients in each layer of U-Net. The $y$-axis on the right denotes the epoch number. Each slice shows the exponential distribution of the gradients. Darker shades: larger gradients. Lighter shades: smaller gradients.

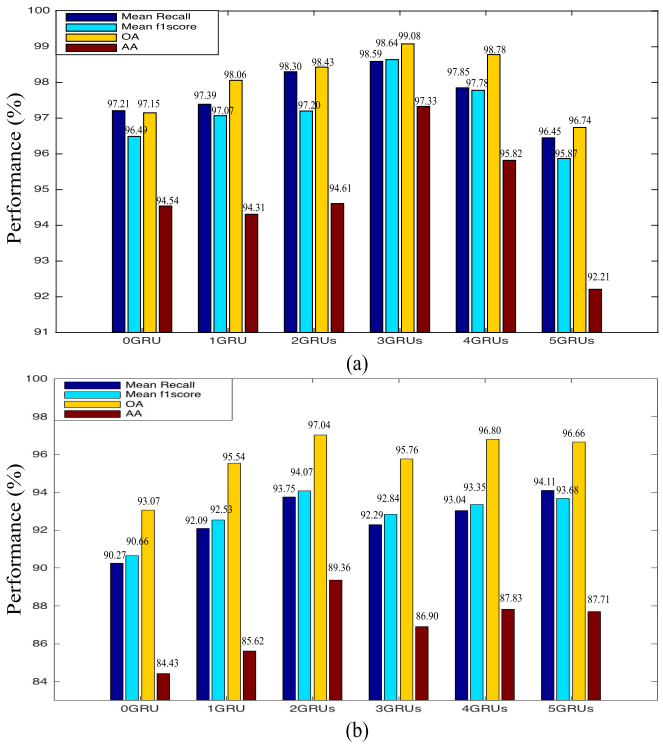



Fig. 7. Impact of the number of GRUs on (a) Pavia University data set (b) Houston University data set.

of GRUs, until the number of GRUs is equal to 3. At this point, Mean Recall = 98.59%, Mean F1 score = 98.64%, OA = 99.08%, and AA = 97.33%. After that, the performance decreases slightly. For the Houston University data set, when the number of GRUs is set to 2, the accuracy reaches its maximum (i.e., Mean F1 score = 94.07%, OA = 97.04%, and AA = 89.36%). Thus, three and two GRUs are suitable for the Pavia University and Houston University data sets, respectively. We performed the following experiments using three GRUs for Pavia University and two GRUs for Houston University.

Furthermore, to explore the impact of the size of the hidden state vector $h_0$ versus the recurrence steps, we compared two cases: two GRUs with each $h_0$ of size 32 and one GRU with $h_0$ of size 64. These two strategies led to, respectively, 93.75% versus 87.94% in Mean Recall, 94.07% versus 88.24% in Mean F1 score, 97.04% versus 87.50% in OA, and 89.36% versus 78.16%, which clearly indicates the superiority of the former. Subsequently, we further explore the impact of the depth of the proposed small network embedded in the GRU architecture. We provide the results obtained by varying the depth in the set {1, 2, 3} in Table II. Note that, whether on small or large data sets, the trend in accuracy change is similar. Accuracy typically increases until depth is equal to 2 and then drops slightly.

After confirming the above parameters, we verify the validity of the proposed geometry-aware loss function constructed using a ResNet. To this end, we compare the proposed loss with the cross-entropy one. In addition, we also used layers 3 and 17 of the VGG19 to calculate the geometry-aware loss. For this experiment, we selected 60% and 10% of the cropped images as the training and validation data for the small Pavia University data set. For the large Houston University data set,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

TABLE II

IMPACT OF THE DEPTH OF THE SMALL NETWORK EMBEDDED IN THE GRU ARCHITECTURE ON PAVIA UNIVERSITY AND HOUSTON UNIVERSITY

| | Pavia University | | | Houston University | | |
|---|---|---|---|---|---|---|
| Performance (%) | depth = 1 | depth = 2 | depth = 3 | depth = 1 | depth = 2 | depth = 3 |
| Mean Recall | 98.59 | **99.20** | 97.65 | 92.29 | **93.94** | 91.25 |
| Mean f1score | 98.64 | **98.97** | 96.48 | 92.84 | **93.85** | 91.42 |
| OA | 99.08 | **99.26** | 97.53 | 95.76 | **96.89** | 94.56 |
| AA | 97.33 | **97.97** | 93.23 | 86.90 | **88.80** | 84.31 |

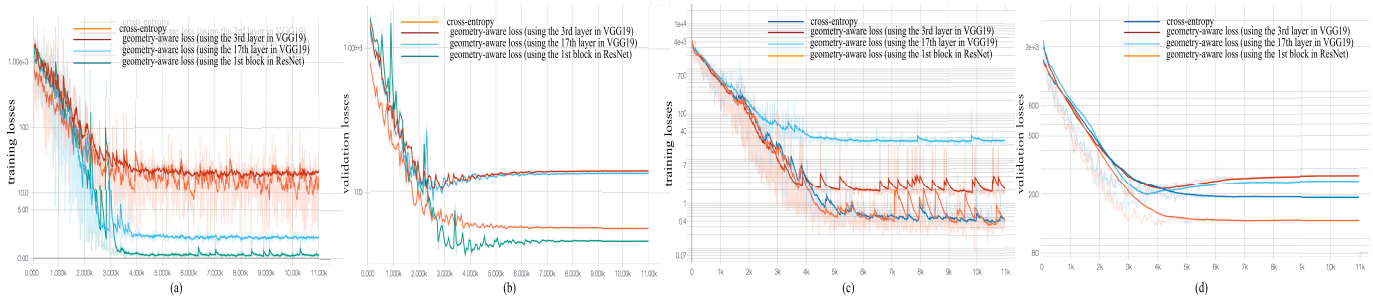



Fig. 8. Loss behavior over the training phase. Training loss and validation loss on (a) and (b) Pavia University and (c) and (d) Houston University.

we selected the training samples according to Fig. 4(c) and considered the test samples as the validation ones. We used stochastic gradient descent with backpropagation to optimize the networks, setting the learning rate to $1e$-2 and using a weight decay of 0.0005. The experimental results on the two data sets are provided in Fig. 8. As shown in Fig. 8(a), all the losses are close to convergence after 3000 epochs. However, after that, the orange curve obtained by the cross-entropy loss still fluctuates. By contrast, the geometry-aware losses calculated by the first block of the ResNet and the third layer of the VGG19 are more stable and reach lower training loss values of 0.05 and 1.87, respectively. The training loss obtained using the 17th layer of the VGG19 cannot decrease to a small value. This further confirms that lower layers are more suitable to describe geometric information. Let us then analyze Fig. 8(b), in which the red and blue loss curves (obtained by VGG19) slowly rise after convergence. This trend indicates that the models are overfitting. Similar conclusions can be drawn from Fig. 8(c) and (d). The validation loss of our method is very stable and does not rise again after convergence, which indicates that no overfitting occurs in the training process on Houston University. Moreover, we evaluate the impact of combing the features from all the GRUs with different weights to make the final prediction. The results of this experiment are provided in Table III, where we define the weights of the three GRUs as $w_1$, $w_2$, and $w_3$. Note that the performance using the last hidden representation only is higher than that using the mixed ones.

To illustrate visually the superiority of the proposed loss, we exploited two models, trained by the geometry-aware loss and cross-entropy loss, respectively, to classify images with complex road textures. The classification maps are shown in Fig. 9. Note that the model using the geometry-aware loss better represents complex geometries. In summary, the proposed geometry-aware ResNet loss is more suitable for the considered data set.

TABLE III

IMPACT BY DIFFERENT WEIGHT COMBINATIONS OF GRUS ON THE HOUSTON UNIVERSITY DATA SET

| | value of $w_1, w_2, w_3$ | | |
|---|---|---|---|
| Performance (%) | $0, 0, 1$ | $1, 1, 1$ | $0.64, 0.8, 1$ |
| Mean Recall | **93.94** | 88.67 | 89.33 |
| Mean f1score | **93.85** | 89.70 | 90.68 |
| OA | **96.89** | 92.65 | 94.26 |
| AA | **88.80** | 81.39 | 83.80 |

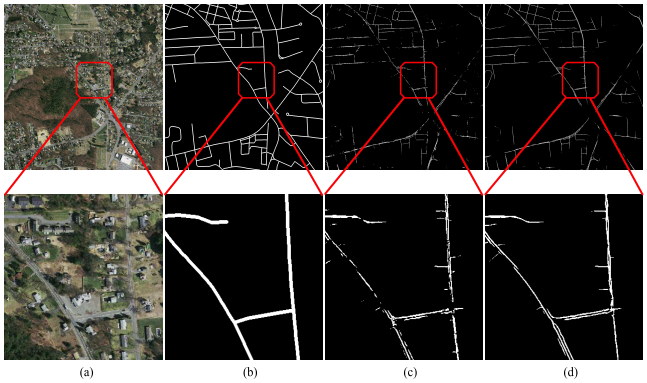



Fig. 9. Visual advantage of the proposed loss on Massachusetts Roads. (a) Representative road region. (b) Ground truth. (c) Result obtained with the cross-entropy loss. (d) Result obtained with our geometry-aware ResNet loss.

### C. Generalization Ability

To verify the generalization ability of our method, we conducted an experiment on the disjoint training/testing splits of the Houston data set. The two splits are considered, as shown in Fig. 4(c) and (d). The one in Fig. 4(c) yields some overlap between the training and testing samples, ensuring that the samples of each class are present in both the training and testing sets. In Fig. 4(d), the training and testing splits have no

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

COMPARISON OF THE NUMBER OF PARAMETERS AND FLOPS OF DIFFERENT METHODS ON DIFFERENT DATA SETS

| Models (Million) | Structure | Pavia University | | Indian Pines | | Houston University | |
|---|---|---|---|---|---|---|---|
| | | Params | FLOPS | Params | FLOPS | Params | FLOPS |
| 3D-CNNs | Conv3D + Dense | 8.22 | 16.44 | 16.62 | 33.23 | 11.77 | 23.54 |
| R-UNet | UNet + RNNs | 4.29 | 2789.76 | 4.32 | 3160.64 | 4.30 | 2958.52 |
| Geo-DRNNs | UNet + RNNs | 7.85 | 1772.92 | 7.88 | 1890.18 | 7.86 | 1823.72 |

TABLE V

COMPARISON OF CLASSIFICATION ACCURACIES (IN PERCENT) PROVIDED BY DIFFERENT METHODS USING 10% TRAINING IMAGES (PAVIA UNIVERSITY)

| Class | No. | SVM | Dictionary learning | | Deep learning | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | OMP | LC-KSVD2 | SdAE | LeNet | SRCNN | 3D-CNNs | SRCL | R-UNet | Geo-DRNNs |
| Asphalt | 6631 | 91.32 | 77.51 | 94.10 | 95.82 | 90.98 | 96.38 | 96.17 | 96.92 | **97.67** | 96.33 |
| Meadows | 18649 | 97.61 | 95.25 | 99.13 | 96.22 | 99.31 | 99.79 | 95.76 | **99.48** | 99.42 | 97.99 |
| Gravel | 2099 | 74.38 | 60.72 | 60.61 | 85.85 | 26.01 | 78.23 | 91.57 | 86.56 | **92.80** | 85.79 |
| Trees | 3064 | 86.43 | 82.95 | 89.19 | 96.12 | 95.82 | **97.68** | 95.66 | 94.52 | 95.88 | 96.46 |
| Metal sheets | 1345 | 87.44 | 99.50 | 99.92 | 98.95 | 99.78 | **100.00** | **100.00** | **100.00** | 99.40 | 99.78 |
| Bare soil | 5029 | 84.14 | 56.98 | 66.20 | 93.04 | 95.46 | 91.74 | 92.54 | 94.97 | **97.13** | 94.80 |
| Bitumen | 1330 | 83.71 | 76.52 | 41.19 | 79.92 | 95.79 | 89.39 | 74.89 | 88.95 | 85.33 | **93.73** |
| Bricks | 3682 | 88.38 | 76.49 | 86.18 | 84.06 | **99.10** | 98.66 | 88.35 | 96.58 | 95.73 | 93.71 |
| Shadows | 947 | 96.95 | 89.91 | 96.83 | 99.47 | **99.79** | 99.47 | 87.75 | 97.15 | 99.15 | 98.93 |
| AA | - | 87.82 | 79.54 | 81.48 | 92.16 | 89.12 | 94.59 | 91.41 | 95.01 | **95.84** | 95.28 |
| OA | - | 91.55 | 83.24 | 88.93 | 93.87 | 93.62 | 96.68 | 93.90 | 96.95 | 97.54 | **98.05** |

TABLE VI

COMPARISON OF CLASSIFICATION ACCURACIES (IN PERCENT) PROVIDED BY DIFFERENT METHODS USING 10% TRAINING IMAGES (INDIAN PINES)

| Class | No. | SVM | Dictionary learning | | Deep learning | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | OMP | LC-KSVD2 | SdAE | LeNet | SRCNN | 3D-CNNs | SRCL | R-UNet | Geo-DRNNs |
| Alfalfa | 54 | 59.03 | 63.89 | 22.22 | 64.81 | 77.78 | 79.62 | 81.48 | 90.74 | **94.44** | 81.47 |
| Corn-no till | 1434 | 78.73 | 61.37 | 83.83 | 76.71 | 88.35 | 91.21 | 84.94 | 93.58 | 94.90 | **94.48** |
| Corn-min till | 834 | 64.31 | 43.07 | 48.59 | 64.63 | 88.12 | 91.01 | 88.97 | 94.12 | 93.28 | **94.76** |
| Corn | 234 | 72.07 | 49.84 | 47.47 | 67.09 | 67.94 | 67.09 | 89.74 | 82.91 | 91.45 | **93.05** |
| Grass/trees | 497 | 88.52 | 82.40 | 83.16 | 85.31 | 87.12 | 90.14 | 96.57 | 97.38 | 97.98 | **98.45** |
| Grass/pasture | 747 | 95.39 | 84.38 | 97.00 | 94.11 | 99.59 | 98.39 | 98.39 | 98.79 | 99.19 | **99.65** |
| Grass/pasture-mowed | 26 | 73.91 | 57.97 | 6.67 | 84.61 | 65.38 | 88.46 | 88.46 | **96.15** | 92.31 | 92.43 |
| Hay-windrowed | 489 | 98.48 | 96.59 | 99.27 | 97.95 | 96.93 | 95.91 | 99.80 | 99.79 | **100.00** | 97.99 |
| Oats | 20 | 38.89 | 24.07 | 2.08 | 60.00 | 85.00 | **95.00** | **95.00** | 90.00 | 40.00 | 40.74 |
| Soybeans-notill | 968 | 69.85 | 60.39 | 70.07 | 79.85 | 86.98 | 87.08 | 80.99 | **94.52** | 94.11 | 92.77 |
| Soybeans-min till | 2468 | 87.56 | 71.69 | 82.21 | 86.30 | 91.97 | 95.78 | 96.88 | **97.24** | 96.27 | 95.90 |
| Soybeans-clean till | 614 | 82.79 | 50.85 | 72.55 | 73.45 | 80.29 | 79.32 | 83.39 | **94.29** | 87.62 | 91.79 |
| Wheat | 212 | 98.07 | 98.60 | 99.44 | 98.58 | 99.53 | 98.58 | **100.00** | **100.00** | **100.00** | 98.26 |
| Woods | 1294 | 95.96 | 91.75 | 97.27 | 95.90 | 97.91 | 98.99 | **98.68** | 98.37 | 98.53 | 98.56 |
| Bldg-grass-tree-drives | 380 | 49.80 | 38.79 | 68.12 | 62.37 | 71.58 | 86.31 | 82.89 | 83.42 | 86.32 | **94.68** |
| Stone-steel towers | 95 | 89.41 | 84.31 | 85.00 | 89.47 | 82.11 | 91.58 | **96.84** | 93.68 | **96.84** | 96.29 |
| AA | - | 77.67 | 66.25 | 67.71 | 80.07 | 85.41 | 89.65 | 91.44 | **94.06** | 91.46 | 91.33 |
| OA | - | 82.93 | 69.58 | 81.33 | 82.93 | 89.91 | 92.22 | 92.05 | 95.58 | 95.35 | **97.77** |

overlap. The results in Fig. 4(d) show that U-Net (composed of U-Net and fully connected layer) achieves an OA = 72.45%, an AA = 47.25%, a Mean Recall = 69.13%, and a Mean F1 score = 65.53%, while the proposed Geo-DRNNs yield an OA = 73.40%, an AA = 47.95%, a Mean Recall = 71.64%, and a Mean F1 score = 66.90%. This suggests that the proposed method generalizes better than U-Net. However, the performance is not as good as the one obtained on the region shown in Fig. 4(c). This is because, in Fig. 4(d), some categories, such as Highway and Railway, are only present in the testing region. Therefore, the trained classifier does not contain any information about these two categories, which will be randomly classified during the testing phase.

### *D. Parameter Analysis of Geo-DRNNs*

In this section, we discuss the number of parameters in the Geo-DRNNs. The parameters of the Geo-DRNNs mainly come from the U-Net and the RNNs. The U-Net ones consist of the kernels and biases of the encoder and the decoder, whose amount is decided by the depth of the U-Net. Similarly, in the RNNs, the kernel and bias sizes are determined by the gates, and the number of parameters of the hidden state is fixed. In other words, once the network is defined, the number of parameters is only affected by the input spatial size and the number of input channels. For the large Houston University data set, we follow the split strategy shown in Fig. 4(c). For the other smaller data sets, we select training samples from the whole image randomly. We report the number of parameters and the floating point operations per second (FLOPS) of different methods on different data sets in Table IV. Note that the 3-D-CNN has a simple structure only including two Conv3D layers and two Dense layers. Because it does not contain a complicated gate structure, it requires fewer floating point operations than the other methods. The R-UNet and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

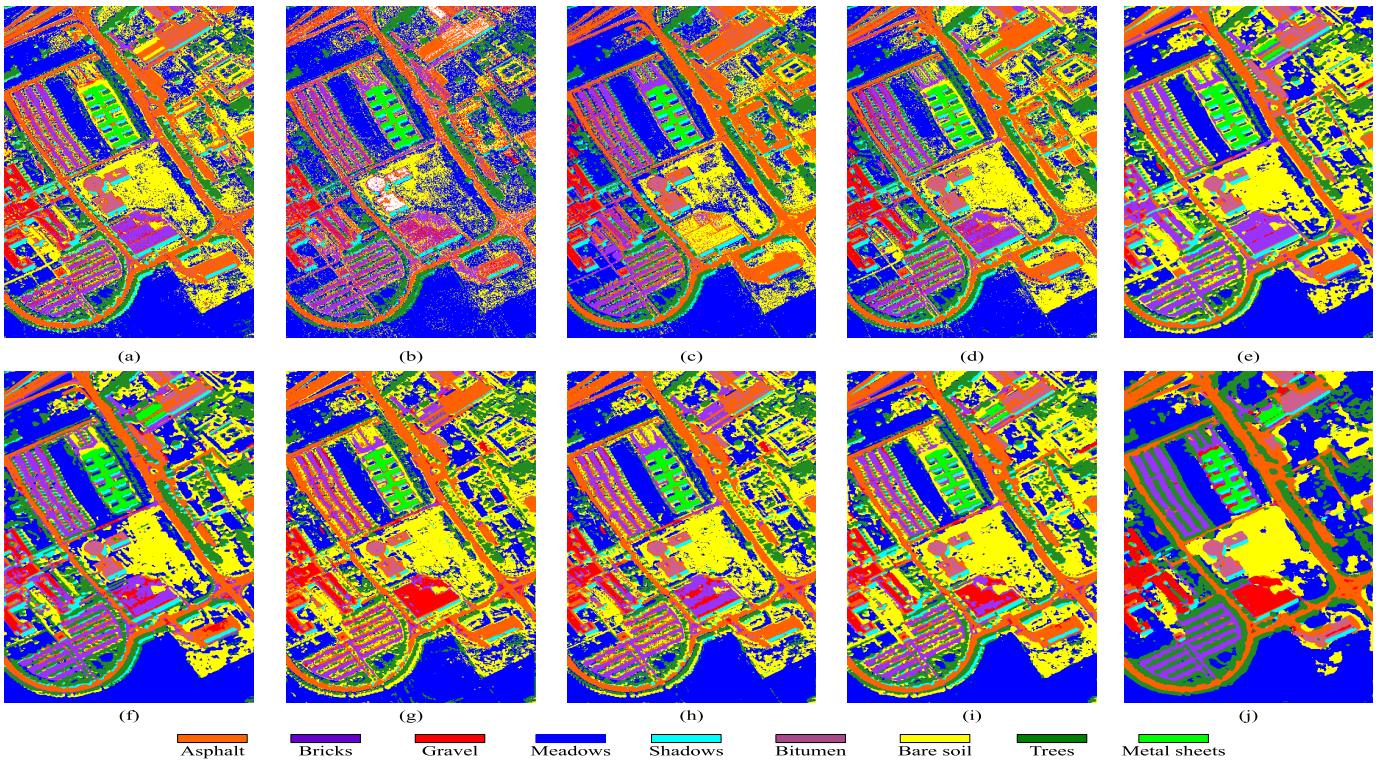



Fig. 10. Visual comparison of the classification maps obtained by different methods for the Pavia University data set. (a) SVM. (b) OMP. (c) LC-KSVD2. (d) SdAE. (e) LeNet. (f) CNN-SR. (g) 3-D-CNNs. (h) SRCL. (i) R-UNet. (j) Geo-DRNNs.

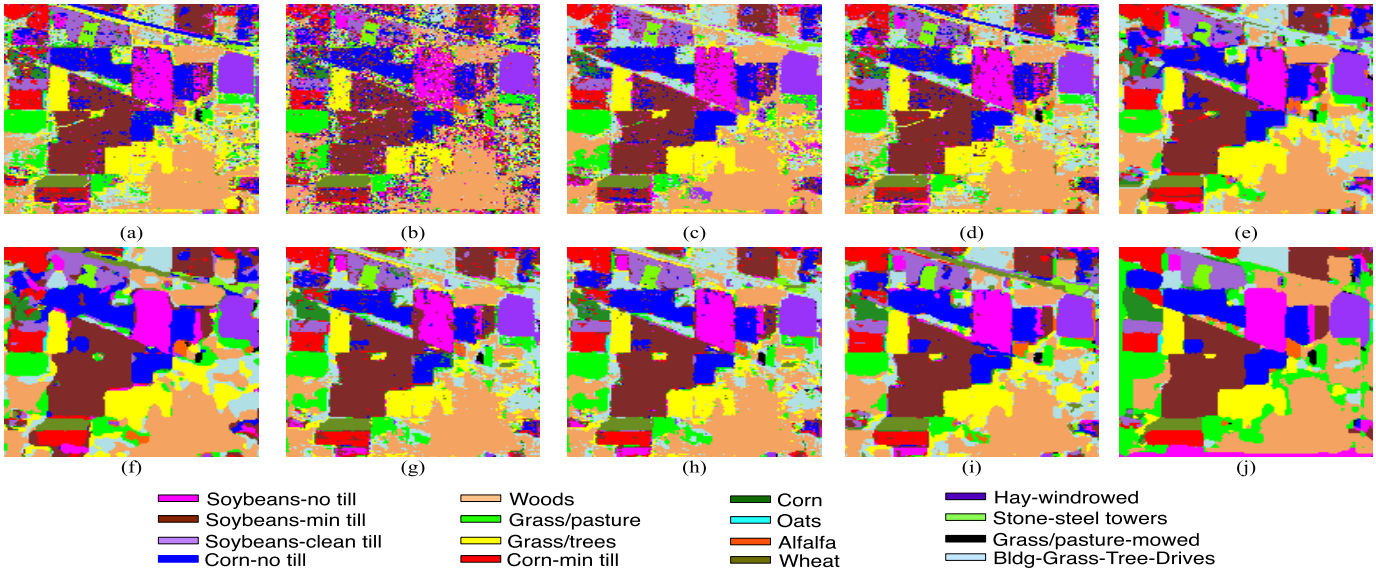



Fig. 11. Visual comparison of the classification maps obtained by different methods for the Indian Pines data set. (a) SVM. (b) OMP. (c) LC-KSVD2. (d) SdAE. (e) LeNet. (f) CNN-SR. (g) 3-D-CNNs. (h) SRCL. (i) R-UNet. (j) Geo-DRNNs.

Geo-DRNN have a similar network structure. Although the Geo-DRNN has more parameters to estimate than the R-UNet, it still benefits from a lower computational cost.

### *E. Comparison With Other Classification Methods*

In this section, we selected SVM, dictionary-based classification methods (OMP [51] and LC-KSVD2 [52]), and deep-learning-based classification methods (SdAE [53], LeNet [54], 3-D-CNNs [12], SRCNN [23], SRCL [23], and R-UNet [55]) as reference methods to verify the advantage of our Geo-DRNNs, especially for scenarios where small amounts of training images are available. We randomly selected 10% cropped images as training set, 10% as validation set, and the remaining 80% as test set. If we crop an image patch for each pixel, the total image patches of the entire image

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

(e.g., the Pavia University data set) will occupy large memory. Therefore, we cropped an image patch every two pixels (i.e., the cropping stride is set to 2) in each row and each column. The parameter setting for the benchmarks can be found in the corresponding references. For the proposed Geo-DRNNs, we set the spatial size of the input patches to $s = 16 \times 16$, the depth of the small network in the GRUs to 2, and exploited the first block of the ResNet to compute the geometry-aware loss. The batch size and the learning rate in the training phase were fixed to 16 and $1e$-2, respectively. The experimental results are reported in Tables V and VI. Analyzing Table V, concerning the Pavia University data set, allows us to conclude that the proposed Geo-DRNNs is clearly beneficial to classify bitumen and bricks. Moreover, the proposed Geo-DRNNs achieve the best overall performance of 95.28% in AA and 98.05% in OA. The Indian Pines data set is more challenging because of its relatively small image size and because it includes fewer pixels. We can see from Table VI that the proposed Geo-DRNNs for the Indian Pines data set achieves the best performance (i.e., AA = 91.33% and OA = 97.77%), with a marked improvement in OA. For illustrative purposes, the classification maps of the considered data sets are presented in Figs. 10 and 11. In summary, the classification maps of the proposed Geo-DRNNs are clearer than those obtained by the baselines.

## V. Conclusion

In this article, we have proposed a novel HSI classification method (Geo-DRNNs). Specifically, we have combined the U-Net with RNNs, so that the former encodes the original HSI, while the latter continuously refines the prediction. In this combination, we have further improved the GRU gates by deepening their convolutional layers, thus increasing its nonlinear representation power. The extracted features are high level and can be used to represent the complex geometries. Furthermore, we have proposed to input the obtained classification results and the corresponding ground truth into a pretrained ResNet to compute a geometry-aware ResNet loss that better encodes the complex geometries. To evidence the superiority of the proposed Geo-DRNNs, we have compared it with the traditional SVMs, dictionary-based classification methods, and the deep learning-based classification methods. Our experimental results have shown that the proposed Geo-DRNNs yield better performance than the baselines and better learns the complex geometries.

Because of the requirement of the U-Net for relatively large input image sizes, cropped image patches take up a large memory. To address this, in our future research, we will explore how to minimize the memory occupation by making better use of the spatial information.

## Acknowledgment

The authors would like to thank Prof. Gamba from the University of Pavia, Italy, for providing the Pavia University data set, and Prof. Landgrebe from Purdue University, USA, for providing the Indian Pines data set.

## References

[1] T. Kam Ho, "The random subspace method for constructing decision forests," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 8, pp. 832–844, Aug. 1998.

[2] J. A. Gualtieri and S. Chettri, "Support vector machines for classification of hyperspectral data," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, vol. 2, Jul. 2000, pp. 813–815.

[3] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[4] N. Audebert, B. Le Saux, and S. Lefevre, "Deep learning for classification of hyperspectral data: A comparative review," *IEEE Geosci. Remote Sens. Mag.*, vol. 7, no. 2, pp. 159–173, Jun. 2019.

[5] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[6] C. Tao, H. Pan, Y. Li, and Z. Zou, "Unsupervised spectral–spatial feature learning with stacked sparse autoencoder for hyperspectral imagery classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 12, pp. 2438–2442, Dec. 2015.

[7] X. Ma, H. Wang, and J. Geng, "Spectral–spatial classification of hyperspectral image based on deep auto-encoder," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 9, pp. 4073–4085, Sep. 2016.

[8] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4700–4708.

[9] W. Hu, Y. Huang, W. Li, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, Jan. 2015, Art. no. 258619.

[10] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4959–4962.

[11] Y. Chen, C. Li, P. Ghamisi, X. Jia, and Y. Gu, "Deep fusion of remote sensing data for accurate classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 8, pp. 1253–1257, Aug. 2017.

[12] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[13] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, 2017.

[14] S. Ji, C. Zhang, A. Xu, Y. Shi, and Y. Duan, "3D convolutional neural networks for crop classification with multi-temporal remote sensing images," *Remote Sens.*, vol. 10, no. 2, p. 75, Jan. 2018.

[15] M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, J. Plaza, A. J. Plaza, and F. Pla, "Deep pyramidal residual networks for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 740–754, Feb. 2019.

[16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[17] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*. [Online]. Available: http://arxiv.org/abs/1409.1556

[18] C. Szegedy *et al.*, "Going deeper with convolutions," 2014, *arXiv:1409.4842*. [Online]. Available: http://arxiv.org/abs/1409.4842

[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[20] G. E. Hinton, S. Osindero, and Y. W. Teh, *A Fast Learning Algorithm for Deep Belief Nets*. Cambridge, MA, USA: MIT Press, 2006.

[21] A. Romero, C. Gatta, and G. Camps-Valls, "Unsupervised deep feature extraction for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 3, pp. 1349–1362, Mar. 2016.

[22] J. Yang, Y. Q. Zhao, and J. C. W. Chan, "Learning and transferring deep joint spectral–spatial features for hyperspectral classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4729–4742, Aug. 2017.

[23] S. Hao, W. Wang, Y. Ye, E. Li, and L. Bruzzone, "A deep network architecture for super-resolution-aided hyperspectral image classification with classwise loss," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4650–4663, Aug. 2018.

[24] R. Mukhometzianov and J. Carrillo, "CapsNet comparative performance evaluation for image classification," 2018, *arXiv:1805.11195*. [Online]. Available: http://arxiv.org/abs/1805.11195

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.

[25] L. Mou, P. Ghamisi, and X. X. Zhu, "Unsupervised spectral–spatial feature learning via deep residual Conv–Deconv network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 1, pp. 391–406, Jan. 2018.
[26] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," 2015, *arXiv:1505.04597*. [Online]. Available: https://arxiv.org/abs/1505.04597
[27] I. Arganda-Carreras *et al.*, "Crowdsourcing the creation of image segmentation algorithms for connectomics," *Frontiers Neuroanatomy*, vol. 9, p. 142, Nov. 2015.
[28] X. X. Zhu *et al.*, "Deep learning in remote sensing: A comprehensive review and list of resources," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 4, pp. 8–36, Dec. 2017.
[29] X. Zhang, Y. Sun, K. Jiang, C. Li, L. Jiao, and H. Zhou, "Spatial sequential recurrent neural network for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 11, pp. 4141–4155, Nov. 2018.
[30] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.
[31] H. Wu and S. Prasad, "Convolutional recurrent neural networks for Hyperspectral data classification," *Remote Sens.*, vol. 9, no. 3, p. 298, Mar. 2017.
[32] Q. Liu, F. Zhou, R. Hang, and X. Yuan, "Bidirectional-convolutional LSTM based spectral-spatial feature learning for hyperspectral image classification," *Remote Sens.*, vol. 9, no. 12, p. 1330, Dec. 2017.
[33] M. Rußwurm and M. Körner, "Multi-temporal land cover classification with sequential recurrent encoders," *ISPRS Int. J. Geo-Inf.*, vol. 7, no. 4, p. 129, Mar. 2018.
[34] P. Benedetti, D. Ienco, R. Gaetano, K. Osé, R. Pensa, and S. Dupuy, "M3Fusion: A deep learning architecture for multiscale multimodal multitemporal satellite data fusion," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 2, pp. 4939–4949, 2018.
[35] L. Mou, L. Bruzzone, and X. X. Zhu, "Learning spectral-spatial-temporal features via a recurrent convolutional neural network for change detection in multispectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 924–935, Feb. 2019.
[36] X. Yang, Y. Ye, X. Li, R. Y. K. Lau, X. Zhang, and X. Huang, "Hyperspectral image classification with deep learning models," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5408–5423, Sep. 2018.
[37] A. Mosinska, P. Marquez-Neila, M. Kozinski, and P. Fua, "Beyond the pixel-wise loss for topology-aware delineation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3136–3145.
[38] C. Ledig *et al.*, "Photo-realistic single image super-resolution using a generative adversarial network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 105–114.
[39] R. M. Rustowicz, R. Cheong, L. Wang, S. Ermon, M. Burke, and D. B. Lobell, "Semantic segmentation of crop type in Africa: A novel dataset and analysis of deep learning methods," in *Proc. Conf. Comput. Vis. Pattern Recognit. (CVPR) Workshops*, Long Beach, CA, USA, Jun. 2019, pp. 75–82.
[40] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar. 1994.
[41] M. Siam, S. Valipour, M. Jagersand, and N. Ray, "Convolutional gated recurrent networks for video segmentation," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2017, pp. 3090–3094.
[42] S. Hochreiter and J. J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 80–1735, 1997.
[43] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," 2014, *arXiv:1406.1078*. [Online]. Available: http://arxiv.org/abs/1406.1078
[44] X. SHI, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-C. WOO, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in *Adv. Neural Inf. Process. Syst.*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, Eds. New York, NY, USA: Curran Associates, 2015, pp. 802–810. [Online]. Available: http://papers.nips.cc/paper/5955-convolutional-lstm-network-a-machine-learning-approach-for-precipitation-nowcasting.pdf
[45] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
[46] V. Mnih, "Machine learning for aerial image labeling," Ph.D. dissertation, Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, 2013.
[47] F. Pacifici, Q. Du, and S. Prasad, "Report on the 2013 IEEE GRSS data fusion contest: Fusion of hyperspectral and LiDAR data [Technical Committees]," *IEEE Geosci. Remote Sens. Mag.*, vol. 1, no. 3, pp. 36–38, Sep. 2013.
[48] W. Liao, A. Pizurica, R. Bellens, S. Gautama, and W. Philips, "Generalized graph-based fusion of hyperspectral and LiDAR data using morphological features," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 3, pp. 552–556, Mar. 2015.
[49] D. Tuia, R. Flamary, and N. Courty, "Multiclass feature learning for hyperspectral image classification: Sparse and hierarchical solutions," *ISPRS J. Photogramm. Remote Sens.*, vol. 105, pp. 272–285, Jul. 2015. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0924271615000234
[50] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep learning classifiers for hyperspectral imaging: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 158, pp. 279–317, Dec. 2019. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0924271619302187
[51] R. Rubinstein, M. Zibulevsky, and M. Elad, "Efficient implementation of the K-SVD algorithm using batch orthogonal matching pursuit," Dept. Comput. Sci., Technion, Haifa, Israel, Tech. Rep., 2008.
[52] Z. Jiang, Z. Lin, and L. S. Davis, "Label consistent K-SVD: Learning a discriminative dictionary for recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 11, pp. 2651–2664, Nov. 2013.
[53] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Mach. Learn. Res.*, vol. 11, no. 12, pp. 3371–3408, Dec. 2010.
[54] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[55] W. Wang, K. Yu, J. Hugonot, P. Fua, and M. Salzmann, "Recurrent U-Net for resource-constrained segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 2142–2151.



**Siyuan Hao** (Member, IEEE) received the Ph.D. degree from the College of Information and Communications Engineering, Harbin Engineering University, Harbin, China, in 2015.

She is a Researcher with the Qingdao University of Technology, Qingdao, China, where she teaches remote sensing and electrical communication. Her research interests focus on hyperspectral imagery processing and machine learning.



**Wei Wang** received the Ph.D. degree from the University of Trento, Trento, Italy, in 2018.

He holds a post-doctoral position at the École polytechnique fédérale de Lausanne (EPFL), Lausanne, Switzerland. His research interests include computer vision, deep learning, and augmented reality, particularly human centered perception, including face and hand analysis.



**Mathieu Salzmann** (Member, IEEE) received the Ph.D. degree from the École polytechnique fédérale de Lausanne (EPFL), Lausanne, Switzerland, in January 2009, under the supervision of Prof. P. Fua.

He is a Senior Researcher at the Computer Vision Laboratory, EPFL, and an Artificial Intelligence Engineer at ClearSpace, Ecublens, Switzerland. Previously, he was a Senior Researcher and Research Leader with the NICTA's Computer Vision Research Group. Prior to this, from September 2010 to January 2012, he was a Research Assistant Professor at TTI-Chicago, Chicago, IL, USA, and from February 2009 to August 2010, a Post-Doctoral Fellow at ICSI and EECS, UC Berkeley, under the supervision of Prof. T. Darrell.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:58:45 UTC from IEEE Xplore. Restrictions apply.